

Endgame Killer: Design and Implementation of a Hybrid Gomoku AI on Triangular Board

Personal Technical Report

LAN Tianwei
MSDM 5002 Final Project
December 11, 2025

1 Introduction

The triangular Gomoku board introduces significant asymmetry and complex connectivity compared to the standard rectangular version. Positions have different numbers of neighbours, threat patterns are harder to enumerate exhaustively, and the winning condition must strictly prohibit overlines (more than 5 in a row). These factors make traditional rectangular-board heuristics ineffective.

Our Endgame Killer AI adopts a classic layered decision architecture: immediate tactical solver (VCT) $\rightarrow$ lightweight pattern detector $\rightarrow$ scoring system $\rightarrow$ Monte Carlo Tree Search (MCTS). This hierarchy ensures that obvious forcing lines are solved deterministically in milliseconds, while non-tactical positions are handed to the probabilistic strategic engine.

In this report I focus exclusively on the components I personally designed and implemented, explain my understanding of MCTS in this context, and highlight the technical challenges I solved in the detector and utility layers.

2 My Understanding and Use of Monte Carlo Tree Search

MCTS is fundamentally an anytime algorithm that converts raw simulation power into decision quality. Instead of trying to prove the value of a position, it estimates it through sampling. This makes it ideal for games with very high branching factors (triangular board has up to 100 legal moves in midgame) and where perfect information does not easily yield short proofs.

In our project, MCTS is not used as a standalone engine but as the final strategic fallback:

- The search tree is heavily biased at the root by the moves returned from `detector.candidates_moves`.
- Leaf evaluation is replaced by a fast kernel-score heuristic (max own threat score – max opponent threat score) instead of random playouts, dramatically reducing variance.
- A strict wall-clock time budget (typically 1–3 seconds) is enforced, after which the move with highest visit count is selected.

Because tactical positions are intercepted earlier by VCT and the detector, MCTS rarely wastes time exploring lines that are already provably winning or losing. This hybrid usage is what gives the AI its “killer” characteristic in the endgame while remaining competent in quiet positions.

3 Major Personal Contributions

3.1 Utility Functions and Game Interface (`utility_func.py`)

I was solely responsible for the entire visualisation and interaction layer:

- Designed the hexagonal-to-pixel coordinate transformation for the triangular grid and implemented accurate click-to-index conversion using geometric reasoning.
- Implemented precise stone drawing with anti-aliased circles and dynamic highlighting (red for latest move, green for winning line).
- Wrote a robust exactly-five-in-a-row detector that explicitly checks both endpoints to forbid overlines — a non-trivial requirement on a triangular board where lines have irregular lengths.
- Implemented the winning-line highlighting routine that correctly steps along all three axes (horizontal, vertical, diagonal).
- Added asynchronous AI move handling using pygame custom events and threading to prevent the GUI from freezing during search.

These components were rewritten multiple times during development to fix subtle coordinate bugs and to achieve smooth 30 fps gameplay.

3.2 Pattern Detector Core Functions (`detector.py`)

I contributed several critical detection functions and their optimisations:

- Implemented `_creates_overline()` that simulates placement and scans all three directions to prevent illegal six-plus lines.
- Wrote `has_nhd()` (has neighbour) with full six-direction checks, used both for proximity bonuses and for pruning irrelevant empty cells.
- Designed and refined the line-generation routine `_board_to_lines()` that correctly inserts boundary sentinels (value 5)
- essential for pattern matching to work at board edges.
- Implemented `weak_type2_moves()` and `type2_moves()` using the pattern-to-moves scanner, and added overline filtering to ensure legality.
- Optimised `candidates_moves()` to return prioritised threat levels (1–4 for forced moves, 99 for normal play), which is the primary early-exit mechanism before MCTS is invoked.

These functions are heavily used not only by the scoring system but also by VCT and the main AI decision chain.

3.3 Architecture-Level Optimisations and Bug Fixing

Throughout the project I acted as the primary debugger and integration engineer:

- Discovered and fixed multiple race conditions in the original synchronous AI implementation by introducing thread-safe event posting.
- Added extensive caching in `score.py` (both raw and kernel scores) to prevent recomputing identical boards hundreds of times per second.
- Ensured consistent board representation (upper triangle = 5, flipped matrix) across all modules.
- Designed the final decision cascade in `endgame_killer.py`: immediate win $\rightarrow$ opponent immediate win $\rightarrow$ VCT proof $\rightarrow$ single forced candidate $\rightarrow$ MCTS.

4 Acknowledgements

I would like to sincerely thank my three teammates for their excellent work on the MCTS engine, VCT solver, scoring propagation, and overall system integration. Their high-quality implementations and timely responses made it possible to focus on the visualisation and detection layers without blocking progress. All members contributed outstanding work and communicated effectively throughout the project.

5 Peer Evaluation

Teammate	Participation Score (0–5)
HUANG, Wenhao	5
XIAO, Shiqin	5
CHEN, Yilin	5